

PROJECT REPORT

Script-Controlled ACL - Restrict Access Based on Field

Project Testing | ServiceNow (Assumed)

S.No	Team Role	Name	Register Number
1	Team Leader	Parimala	ASALUAU020224128026
2	Team Member	Rajeswari	ASALUAU020224128029
3		Priyanka	ASALUAU020224128028
4		Rathika	ASALUAU020224128030
5		Mounika	ASALUAU020224128025

1. Abstract

Access Control List (ACL) is a security mechanism used to control who can access data in an application. In ServiceNow, ACL rules determine whether a user can read, create, update, or delete records. This project focuses on a Script-Controlled ACL that restricts access based on field values. A script checks the logged-in user's details against values stored in a record and grants access only when the defined conditions are satisfied. Testing includes positive, negative, functional, and security scenarios to verify that unauthorized access is prevented.

2. Introduction

Different users require different levels of access to application data. For example, employees may need to view records belonging to their own department, while only the assigned user may update a record. A Script-Controlled ACL evaluates such conditions dynamically and helps protect confidential information.

3. Problem Statement

Role-based permissions alone may be too broad when users with the same role should access different records. The project addresses this by checking record-specific field values before allowing an operation.

4. Objectives

- Implement a Script-Controlled ACL.
- Restrict record access based on specific field values.
- Allow authorized users to access permitted records.
- Prevent unauthorized viewing and modification.
- Test read and write permissions using positive and negative cases.
- Identify defects and verify that access restrictions work consistently.

5. Scope

- Record-level access restrictions.
- Field-based validation using Department, Assigned To, or Status.
- Read and write ACL testing.
- Testing through forms, lists, and supported APIs.

Security and regression testing.

6. Existing System

In a basic system, access may be assigned mainly through roles. This can allow users to access more records than they need. Record-specific restrictions may require additional logic, and manual permission management can become difficult.

7. Proposed System

The proposed system uses a Script-Controlled ACL to compare user information with record field values. If the condition is satisfied, access is granted; otherwise, access is denied. For example, a user may read records from their own department and update records assigned to them.

8. Requirements

Hardware

Desktop or laptop computer

4 GB RAM minimum (recommended)

Internet connection for a cloud instance

Software

ServiceNow instance

Web browser

JavaScript

Test user accounts

These are suggested requirements for a ServiceNow implementation. Exact requirements depend on the instance and project environment.

9. System Architecture

1. User logs in
2. User requests a record or operation
3. ACL evaluates roles, conditions, and script
4. Condition satisfied: access granted
5. Condition not satisfied: access denied

10. Implementation

10.1 Sample table and fields

Field	Description
u_employee_name	Employee name
u_department	Department reference
u_assigned_to	Assigned user reference
u_status	Record status
u_access_level	Confidentiality level

10.2 Sample test users

User	Department	Purpose
Arun	IT	Authorized user for IT records
Priya	HR	User from another department
Ravi	IT	Same department, different owner

10.3 Department-based Read ACL

Create a record-level Read ACL for the relevant table. Enable Advanced scripting and use the correct field name for the custom table.

```
answer = false;  
if (gs.getUser().getDepartmentID() &&  
    current.u_department.toString() ==  
    gs.getUser().getDepartmentID()) {  
    answer = true;  
}
```

Explanation: The script denies access by default, obtains the logged-in user's department ID, and compares it with the record's department reference. Access is granted when the values match.

10.4 Assigned To-based ACL

```
answer = false;  
if (current.u_assigned_to.toString() ==
```

```

gs.getUserID()) {
    answer = true;
}

```

This example allows access when the record's Assigned To field matches the logged-in user's ID. Use it only for the intended ACL operation and verify field names.

10.5 Status-based Write ACL

```

answer = false;
if (current.u_status.toString() == 'open') {
    answer = true;
}

```

This example permits the operation only when the stored choice value is 'open'. Confirm the actual stored value in the instance; it may differ from the displayed label. Important: Configure Read and Write ACLs separately when their rules differ. Validate inherited ACLs, role requirements, field names, reference values, and platform evaluation behavior. A script should be tested in a non-production instance.

11. Testing Methodology

Testing type	Purpose
Functional testing	Verify access rules behave as specified
Positive testing	Confirm authorized users can access permitted records
Negative testing	Confirm unauthorized users are blocked
Boundary testing	Check empty or missing field values
Security testing	Check for unauthorized access paths
Regression testing	Ensure existing permissions still work
Integration testing	Verify behavior through forms, lists, and APIs

12. Test Cases

Assumptions: Arun belongs to IT, Priya belongs to HR, record R1 belongs to IT, and record R2 belongs to HR. The following are expected results; actual results must be recorded after execution.

ID	Scenario	Expected result
TC01	Arun reads IT record R1	Access granted

TC02	Priya reads IT record R1	Access denied
TC05	User has no department	Access denied
TC06	Record has no department	Access denied
TC07	Unauthorized user opens restricted record directly	Access denied
TC08	Unauthorized user requests restricted record through a supported API	Access denied
TC09	Assigned user updates a record under owner-based Write ACL	Access granted
TC10	Different user updates that record	Access denied

13. Test Execution Procedure

6. Log in to a non-production ServiceNow instance as an administrator.
7. Create the custom table and required fields.
8. Create test users and assign departments.
9. Insert sample records with different field values.
10. Configure the Read ACL and add the script.
11. Configure a separate Write ACL if required.
12. Impersonate an authorized user and test permitted records.
13. Impersonate an unauthorized user and test restricted records.
14. Test forms, lists, and supported APIs.
15. Review ACL evaluation details and record actual results.
16. Fix defects and rerun failed tests.

14. Expected Results

Scenario	Expected outcome
User and record departments match	Read allowed

Departments differ	Read denied
Record department is missing	Read denied
User is not assigned to record	Write denied under owner-based rule
User is assigned to record	Write allowed under owner-based rule

15. Defect Reporting

Field	Sample value
Defect ID	BUG-001
Title	Unauthorized user can access a restricted record
Module	Access Control
Severity / Priority	High / High
Steps	Log in as HR user and open an IT record
Expected result	Access denied
Actual result	Record is accessible (sample defect)
Status	Open

Possible causes include an incorrect script, a broader ACL granting access, incorrect field configuration, or another applicable security rule.

16. Advantages

- Improves data security and confidentiality.
- Prevents unauthorized record access.
- Supports dynamic field-based restrictions.
- Reduces the risk of unauthorized updates.
- Enables granular permissions beyond role-only checks.

17. Limitations

- Incorrect scripts may block legitimate users.
- Multiple ACL rules can make troubleshooting complex.

Reference fields must be compared correctly.

Field-name changes may require script updates.

All relevant access paths must be tested.

18. Future Enhancements

Combine department and role-based conditions.

Add ownership and access-level checks.

Test Create and Delete permissions.

Automate regression tests.

Add audit reports and automated test-data generation.

19. Conclusion

The Script-Controlled ACL project demonstrates how access to application records can be restricted based on specific field values. By comparing the logged-in user's information with values stored in a record, the system can grant access to authorized users and deny access to unauthorized users. Testing validates positive and negative scenarios, checks different ACL operations, and helps identify security issues. Proper implementation and testing improve data confidentiality, integrity, and controlled access.